LECTURE 14

Background Jobs and Task Queues

Backend Engineering from First Principles

Detailed study notes • Architecture • Reliability • Interview preparation

Backend Engineering from First Principles — detailed study notes

Topic: Background tasks, task queues, producers, consumers, retries, task types, and production design.

1. What is a background task?

A **background task** is a piece of code that runs outside the normal client–server request–response lifecycle.

In a synchronous request, the client waits while the backend performs all required work and returns a response. Some operations do not need to finish before the user receives an acknowledgement. These can be moved to a separate process and completed asynchronously.

Examples:

- Sending verification, welcome, password-reset, and notification emails.
- Resizing, encoding, or otherwise processing images and videos.
- Generating PDF or email reports.
- Sending mobile push notifications.
- Cleaning up expired or abandoned sessions.

Why move work out of the request?

Suppose a user signs up. The backend validates the input, creates the account, generates a verification token, and sends an email through an external email provider.

If the provider is slow or unavailable, synchronous email sending can:

- Delay the signup response.
- Cause the entire signup API to fail if the email error is allowed to propagate.
- Create a misleading experience if signup succeeds but the email silently fails while the UI says it was sent.

Instead, the backend can enqueue an email task and return promptly. A worker sends the email independently and can retry if the provider temporarily fails.

Important: returning after enqueueing means the task has been accepted for processing; it does not mean the email has already been delivered.

2. Synchronous vs asynchronous workflow

Synchronous flow

```
Client
  |
  | POST /signup
```

```
  v
Backend API
  |-- validate input
  |-- create user in database
  |-- generate verification token
  |-- call external email provider  <--- slow/failure-prone dependency
  |
  v
HTTP response
```

The API response is coupled to the email provider's latency and availability.

Background-job flow

```
Client
  |
  | POST /signup
  v
Backend API (producer)
  |-- validate input
  |-- create user
  |-- create verification token
  |-- serialize email-task payload
  |-- enqueue task
  |
  +------------------------------> Queue / Broker
  |
  v
Return HTTP response quickly

Queue / Broker
  |
  | worker receives task
  v
Email worker (consumer)
  |-- deserialize payload
  |-- run registered handler
  |-- call email provider
  |-- acknowledge success OR report failure
  v
Email delivered (or retried / moved to failure handling)
```

The API does not wait for the external provider to finish. The worker performs the slow operation separately.

3. Task queue architecture

A **task queue** is a system for managing and distributing background jobs. It is the mechanism that safely hands work from application code to separate worker processes.

The core parts are:

Component	Responsibility
Producer	Creates a task and places it in the queue

Component	Responsibility
Task payload	Carries the information needed to execute the task
Queue / broker	Temporarily stores tasks until workers can process them
Consumer / worker	Retrieves tasks and executes their handlers
Handler	The actual code that performs a particular task
Acknowledgement (ACK)	Signals that a task was processed successfully

Producer

The producer is application code that creates a task. It gathers the information the worker will need and serializes it into a supported format, often JSON.

Example payload:

```
{
  "type": "send_verification_email",
  "userId": "u_123",
  "email": "person@example.com",
  "firstName": "Asha",
  "verificationUrl": "https://example.com/verify?token=..."
}
```

The enqueue operation is commonly called **enqueueing** or **ENQ**.

Queue / broker

The broker holds tasks until workers are ready. It decouples the speed at which tasks are created from the speed at which they are processed.

A queue is like a backend to-do list:

- Producers add work.
- Workers take work.
- The broker coordinates delivery and task state.

Consumer / worker

A consumer is a program running in a separate process (or another execution unit, depending on the technology). It retrieves tasks, deserializes the payload into native data structures, and invokes the registered handler.

For example:

- JSON becomes a Python dictionary.
- JSON becomes a JavaScript object.

- JSON becomes a Go struct, after decoding into the declared type.

The worker is configured with the queue it consumes and the task types or handlers it knows how to execute.

Dequeueing

Removing or retrieving a task from a queue is called **dequeueing** or **DEQ**.

Acknowledgement

After successful processing, the worker sends an acknowledgement to the broker. The broker can then mark the task complete and remove it from active queue processing.

If the worker crashes or never acknowledges the task, the queue's delivery and timeout rules determine whether the task becomes available again.

4. Serialization and deserialization in job processing

A producer and worker may be separate processes, machines, or even codebases. They need a common representation for task data.

Typical flow:

- Producer builds a task payload.
- Producer serializes the payload (for example, to JSON).
- Broker stores/delivers the serialized message.
- Consumer deserializes it into native values.
- Handler uses the resulting data.

Keep task payloads explicit and stable. Prefer identifiers and the minimum required data over huge, fragile object snapshots. If a worker needs current database state, it can often fetch it using a stable ID.

5. Retries and exponential backoff

A background task can fail because:

- An external service is temporarily unavailable.
- A network request times out.
- A worker crashes.
- A transient database or infrastructure error occurs.

A task framework can retry failed work rather than making the user repeat the original API request.

Exponential backoff

Exponential backoff increases the delay between successive retries.

Example schedule:

Attempt after failure	Wait before next retry
First failure	1 minute
Second failure	2 minutes
Third failure	4 minutes
Fourth failure	8 minutes

A maximum retry count should be configured. The exact schedule and retry semantics depend on the queue framework and application configuration.

After the retry limit is reached, systems commonly stop automatic retries and expose the failed task for inspection or manual recovery. Many systems use a dead-letter or failed-job mechanism, though the lecture does not prescribe one specific implementation.

Do not retry every error blindly. Validation errors and other permanent failures usually need different handling from temporary network failures.

6. Common use cases

6.1 Email delivery

- Verification email after signup.
- Welcome email after successful verification.
- Password-reset email.
- Notifications triggered by user activity.

6.2 Image and video processing

A user uploads media. The backend can acknowledge the upload and process it asynchronously:

- Resize images into multiple dimensions.
- Encode video into multiple formats or resolutions.
- Optimize assets for different devices and network conditions.

6.3 Report generation

Enterprise applications may generate daily, weekly, monthly, or annual reports. Report creation can involve collecting data, rendering HTML/PDF, and sending the result. This is suitable for scheduled or background execution.

6.4 Push notifications

A backend generally stores a device token or registration identifier. To deliver a push notification, it calls the relevant platform push service (for example, Apple's or Google's service). That external call can be performed by a worker.

The backend does not directly deliver a notification to the phone's notification panel; the platform push service participates in delivery.

7. Queue technologies and libraries mentioned

The lecture gives examples across ecosystems:

Ecosystem / role	Examples mentioned
Python task framework	Celery
Node.js task framework	BullMQ
Go task library	Asynq
Message broker	RabbitMQ
In-memory data store / pub-sub option	Redis
Managed cloud queue	Amazon SQS

These products are not interchangeable in every detail. Their delivery guarantees, persistence, ordering, retry configuration, and operational requirements differ. Choose based on the workload and required guarantees.

8. Visibility timeout

A **visibility timeout** is a period during which a delivered task is treated as being in progress and is hidden from other workers.

Typical sequence:

- Worker receives a task.
- The broker starts the visibility/in-progress timeout.
- Worker processes the task.
- Worker acknowledges success before the timeout.

- Broker marks the task complete.

If the worker crashes, hangs, or fails to acknowledge before the timeout, the broker can make the task available to another worker. This helps prevent a task from being permanently lost merely because a worker disappeared.

For long-running work, the timeout must be designed carefully. Some systems support extending or renewing the visibility period while processing.

Consequence: a task may be delivered more than once. Workers should therefore be designed to tolerate duplicate execution.

9. Types of background tasks

9.1 One-off tasks

Triggered by a specific event and usually executed once.

Examples:

- Send a verification email after registration.
- Send a welcome email after verification.
- Send a password-reset email.
- Notify a user when a message arrives.

9.2 Recurring tasks

Run periodically according to a schedule.

Examples:

- Daily, weekly, or monthly reports.
- Cleanup of expired or abandoned sessions.
- Periodic maintenance tasks.

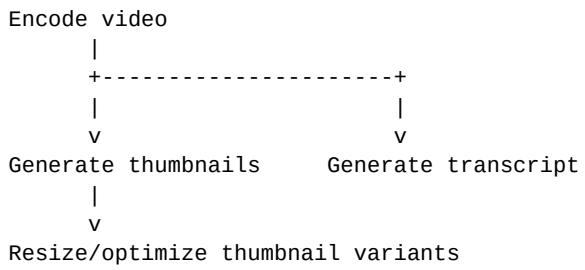
A **cron job** is a common scheduling mechanism. Some task frameworks also provide scheduled or recurring job features.

9.3 Chain tasks

A chain expresses dependencies between tasks: a child task starts only after its prerequisite task succeeds.

Example: processing an uploaded course video.

```
Video uploaded
      |
      v
```



The transcript and thumbnail-generation tasks can run in parallel after encoding if neither depends on the other. Thumbnail resizing depends on thumbnail generation.

9.4 Batch tasks

A batch involves many related units of work, often initiated by one operation or schedule.

Example: account deletion:

- API accepts the deletion request.
- Backend enqueues a deletion workflow and returns.
- Worker finds the user's projects and related entities.
- Worker removes or detaches related records according to product rules.
- Worker deletes profile data and stored assets.
- Worker deletes the account and may send a confirmation email.

Another example is generating and sending thousands of weekly reports to many users.

Distinction: a batch is about processing many items or coordinating a large workload. A chain is about dependency/order between tasks. A real workflow can use both.

10. Production design considerations

10.1 Idempotency

A task is **idempotent** when executing it multiple times has the same intended effect as executing it once.

Why it matters:

- A worker may complete an operation but crash before acknowledging it.
- The broker may redeliver the task.
- A retry may run the same handler again.

Example: a deletion task should safely handle a record that has already been deleted. A payment or email workflow needs stronger duplicate protection because repeating the side effect may be harmful.

Useful techniques:

- Use a stable task or operation ID.
- Store processed operation IDs where appropriate.
- Use unique constraints or conditional updates.
- Make state transitions explicit.
- Use provider-supported idempotency keys for external side effects when available.

Important nuance: a database transaction can make a group of database changes atomic, but it does not automatically make external API calls reversible or make the entire distributed workflow idempotent.

10.2 Error handling

- Catch and classify failures.
- Log enough context to debug the task.
- Retry transient failures.
- Avoid retrying permanent failures indefinitely.
- Define what happens after retries are exhausted.
- Preserve task state and error details for investigation.

10.3 Monitoring

Track:

- Queue depth / queue length.
- Number of successful tasks.
- Number of failed tasks.
- Retry counts.
- Task execution duration.
- Age of the oldest waiting task.
- Worker health and availability.
- Main failure reasons.

The lecture mentions Prometheus and Grafana as examples of metrics and monitoring tools.

10.4 Horizontal scaling

If task volume increases, add more worker instances. This is **horizontal scaling** of consumers.

Monitor throughput and queue growth: adding workers is useful only if the broker, database, external services, and downstream dependencies can handle the additional load.

10.5 Ordering

Some workloads require tasks to execute in a particular order. Confirm that the selected queue technology and configuration provide the ordering semantics the workflow needs. Multiple workers and retries can complicate global ordering.

10.6 Rate limiting

Workers that call external APIs must respect provider rate limits and avoid overwhelming downstream services. Apply concurrency limits, throttling, or rate limiting as appropriate.

11. Best practices

- **Keep tasks small and focused.** A task should have one clear responsibility.
- **Break up long-running tasks.** Split large work into smaller steps or chunks.
- **Use chains when dependencies exist.** Run independent child tasks in parallel where safe.
- **Make handlers idempotent.** Assume duplicate delivery is possible.
- **Implement robust error handling and logging.** Background failures do not automatically appear in the original API response.
- **Monitor queue length and worker health.** Alert when backlog or failures exceed acceptable thresholds.
- **Set sensible retry limits and backoff.** Avoid retry storms.
- **Protect downstream services.** Apply rate limits and concurrency controls.
- **Design task payloads carefully.** Include the information needed to process the job, and version payload schemas when they may change.
- **Expose task status when the user needs it.** For long-running work, return a job ID and provide a status endpoint or notification mechanism.

12. API response semantics for asynchronous work

When an API enqueues a job, distinguish acceptance from completion.

A common pattern is:

- `202 Accepted`: the request has been accepted for processing, but processing is not complete.
- Return a job/task identifier.
- Provide a way to query status, or notify the user when complete.

The lecture's signup illustration mentions success responses such as 200 or 201. In API design, `202 Accepted` is often the clearest status when the main requested operation is still pending in the background. Choose the response based on the resource and API contract.

Example:

```
POST /reports

{
  "jobId": "job_789",
  "status": "queued"
}
```

Later, the client can check:

```
GET /jobs/job_789

{
  "jobId": "job_789",
  "status": "completed",
  "resultUrl": "/reports/report_456"
}
```

13. Quick comparison

Concept	Meaning
Background task	Work executed outside the request–response lifecycle
Producer	Creates and enqueues work
Queue / broker	Holds and distributes tasks
Consumer / worker	Retrieves and executes tasks
Handler	Code that performs the task
Enqueue (ENQ)	Add task to queue
Dequeue (DEQ)	Retrieve task from queue
ACK	Acknowledge successful processing
Retry	Attempt failed work again
Exponential backoff	Increase delay between retries
Visibility timeout	In-progress window before an unacknowledged task can be redelivered
Idempotency	Repeated execution has the same intended effect
One-off task	Event-triggered single task
Recurring task	Scheduled periodic task
Chain task	Tasks linked by dependencies
Batch task	Large group of related work

14. Interview questions and answers

Q1. What is a background job?

A unit of work executed outside the normal request–response lifecycle so the API can respond without waiting for slow, non-critical processing.

Q2. Why should email sending be asynchronous?

Email delivery depends on an external provider that may be slow or temporarily unavailable. Enqueueing the email task keeps signup responsive and allows retries independently.

Q3. What are producer, broker, and consumer?

The producer creates and enqueues tasks; the broker stores and distributes them; the consumer retrieves and executes them.

Q4. What is the difference between enqueue and dequeue?

Enqueue adds a task to the queue. Dequeue retrieves a task for processing.

Q5. What is task acknowledgement?

A signal from the worker to the broker indicating that processing succeeded, allowing the broker to mark the task complete.

Q6. What happens if a worker crashes after receiving a task?

If the broker does not receive an acknowledgement within the configured timeout, it may make the task available again. This can cause duplicate delivery, so handlers should be idempotent.

Q7. Explain exponential backoff.

It increases the delay between retries, for example 1, 2, 4, and 8 minutes, reducing pressure on a temporarily failing dependency.

Q8. What is visibility timeout?

The period during which a delivered task is considered in progress and hidden from other workers. If it is not acknowledged in time, it may be redelivered.

Q9. What is idempotency and why is it important?

It means repeated execution has the same intended effect as one execution. It protects systems from duplicate delivery and retries.

Q10. Chain task vs batch task?

A chain represents dependency between tasks; a batch represents a large set of related work. A workflow may contain both.

Q11. How do you scale background processing?

Increase worker capacity horizontally, while monitoring queue depth, processing time, broker health, and downstream limits.

Q12. What metrics should be monitored?

Queue depth, oldest task age, throughput, success/failure counts, retry counts, execution duration, and worker health.

Q13. Should every task be retried?

No. Retry transient errors; permanent errors such as invalid input need correction or terminal failure handling. Retries should be bounded.

Q14. Why keep tasks small?

Small tasks are easier to retry, debug, monitor, distribute, and scale. A failure in one focused step does not require repeating unrelated work.

15. Practical design checklist

- [ ] Is this work safe to perform after the API response?
- [ ] Does the API clearly distinguish accepted work from completed work?
- [ ] Is the task payload sufficient and versionable?
- [ ] Is there a clear producer, queue, worker, and handler?
- [ ] Are ACK and timeout semantics understood?
- [ ] Can duplicate delivery happen, and is the handler idempotent?
- [ ] Are transient and permanent errors distinguished?
- [ ] Are retries bounded and backed off?
- [ ] Is there a failed-task inspection/recovery path?
- [ ] Are queue depth, task age, failures, and worker health monitored?
- [ ] Can workers scale horizontally without overloading dependencies?
- [ ] Are ordering and rate limits handled where required?

16. Last-minute revision sheet

- Background job = work outside the request–response lifecycle.

- Producer creates and enqueues; broker stores; consumer/worker dequeues and executes.
- Payload is serialized by the producer and deserialized by the worker.
- ACK tells the broker that processing succeeded.
- Missing ACK plus visibility timeout can lead to redelivery.
- Retries help with transient failures; exponential backoff increases the wait.
- One-off = event-triggered; recurring = scheduled; chain = dependency; batch = many related tasks.
- Idempotency is essential because tasks may run more than once.
- Monitor queue length, oldest task age, failures, retries, duration, and worker health.
- Keep tasks focused, bounded, observable, and safe to retry.

Notes are based on the supplied Lecture 14 transcript, with clearly labeled practical backend/API clarifications added where useful.